4. 给我一个普通 module，是怎么开始写 FPV

1. 先看 RTL，别看 FSM 几个状态，哪些 MUX，几级 pipeline，哪些寄存器是某一时钟什么；
2. 先写 CEX 最容易的 property，把 property 和 baseline 先从初始环境里跑起来；
3. 再写所有的状态都有的接口；
4. 对 cover point：把所有的 cover 都跑一遍，确保不是因为 spec 定义不完整导致不合理的覆盖回来的每个条件；
5. 加 assumption：假设环境条件；
6. 加 cover：让 cover 能够逐渐展开；
7. 视一些 CEX 分类原因，CEX 分为三类—RTL bug、property 写法问题、assumption 不够。你先理解。

Ai_p

局部边界：CEX 会用 1 -> 当时应该是，但 delay_cnt 计数回绕，
复位刚释放第一拍这些边界，它没想到。

优先级冲突：RTL 里 rxiqc > ifcnq 的 else-if 优先级，
它写在 property 里加 rxiqc 后就有（sdm_frac/int_update_ifcnq 两条）。
Spec 的工程含义：ckref 要求，平常只包住 calib，
它写了时序断言但漏了复位弱一点的延迟逻辑
(kband_ckref 两条 CEX)。

开场台词（默写）

我们项目里有一套 CoverCheck 自动化 flow。unreachable 分析要对每个模块单独匹配时钟复位、写脚本、排队提交、收集结果，模块多了重复劳动大，容易配错约束。这套 flow 把它拆成生成/批量执行/汇总三段，并封装成 Claude Code 的 skill，让 agent 按流程驱动，关键判断留给人。

必须人接管的四点（SKILL.md 规定）

- Dry-run 后确认 REVIEW、NOCLK 模块的时钟复位是否正确。
- 多时钟模块提供真实频率带，约束是否符合设计值。
- 全量批跑前确认覆盖已有结果前确认。
- 汇总后判断 inconclusive 要不要加大超时重跑，以及 unreachable 是否合理。

设计逻辑六点（放心讲）

1. **问题定义**——unreachable 分析的瓶颈不在工具，而在外围的重复配置。
2. **模板固定、变化约束**——每个模块只有时钟复位 / scan_mode 不同，其余和参考流程一致。
3. **推断不确定就报告，不静默猜**——错误的复位约束会得出只会误导的错误开案。
4. **三段解耦**——生成 / 执行 / 汇总互不调用，靠 run_dirs.txt 衔接，出错不用重跑 license。
5. **资源纪律写进代码**——并行数硬编码为 2（license 上限），已有 work_all 不覆盖输出 --force。
6. **skill 化的分工**——确定性的事全写成脚本；SKILL.md 只写流程、规则，以及"哪里必须停下来问人"。

点	AHB	AXI
核心特点	地址和数据可以流水化	读写完全分离、多个通道独立握手
握手	主要看 `HREADY`	每个通道都是 `VALID && READY`
读写	共用一套总线传输逻辑	读、写可同时进行
性能	较高，但相对简单	更高，适合 CPU、DDR、DMA
验证重点	地址/数据流水关系、`HREADY` 插入等待、Burst	五通道握手、读写乱序、Burst、ID 对应关系

规则	含义
`VALID` 不能依赖 `READY`	发送方一旦准备好数据，就必须主动拉高 VALID，不能等对方 READY
`VALID=1 && READY=0`	当前信息必须保持稳定
`READY` 可以先拉高，也可以后拉高	接收方可随时表示是否能够接收
handshake 后可发送下一笔	同一个通道可以每拍完成一笔传输

地址通道 AW、AR 还要交代“怎么传”：

通道	干什么	主要信号
AW：写地址	主机告诉从机：**往哪写**	`AWADDR`：写入地址
W：写数据	主机告诉从机：**写什么**	`WDATA`：写入数据；`WSTRB`：哪些字节要写；`WLAST`：这是最后一拍
B：写响应	从机告诉主机：**写成功了吗**	`BRESP`：写操作的结果；`BID`：回复的是哪笔写操作
AR：读地址	主机告诉从机：**从哪读**	`ARADDR`：读取地址
R：读数据	从机告诉主机：**读到了什么**	`RDATA`：读出的数据；`RRESP`：读取是否出错；`RLAST`：这是最后一拍；`RID`：属于哪笔读操作

信号	含义	例子
`AWID / ARID`	给这笔操作编个号	ID=2，返回时用 BID/RID 对应它
`AWLEN / ARLEN`	一共传多少拍	LEN=3 → **传4拍**
`AWSIZE / ARSIZE`	每拍传几个字节	SIZE=2 → **每拍4字节**
`AWBURST / ARBURST`	每拍地址怎么变	INCR → 地址依次递增

```
AWLEN = 0  → 1 beat
AWLEN = 3  → 4 beats
AWLEN = 15 → 16 beats
```

地址类型：FIXED / INCR / WRAP

比如：**从 `0x1000` 开始，连续写4个数据，每个4字节。**

- **AW**：交代起始地址 `0x1000`、4拍、每拍4字节、地址递增。
- **W**：依次发送4个数据，第4个带上 `WLAST=1`。
- **B**：全部接收后，返回一次写响应。

先记住：AW 管往哪写，W 管写什么，B 管写的结果；AR 管从哪读，R 管读回来的数据。

AHB-Lite

信号	来源	含义
HCLK / HRESETn	时钟 / 复位源	时钟 / 低有效复位
HADDR	Master	访问地址
HWRITE	Master	**1写、0读**
HWDATA / HRDATA	Master / Slave	写入数据 / 读回数据
HSEL	地址译码器	选中哪个 Slave
HTRANS	Master	**00 IDLE**：空闲；**01 BUSY**：暂停；**10 NONSEQ**：首拍或单次；**11 SEQ**：后续拍
HSIZE	Master	每拍字节数 = 2^HSIZE；**0→1B，1→2B，2→4B**
HBURST	Master	单次或突发传输、突发长度及地址变化方式
HREADY	总线返回	**1：当前数据阶段完成；0：等待**
HREADYOUT	Slave	本 Slave 的完成信号，经总线选择后形成 HREADY
HRESP	Slave	**0：正常；1：错误**

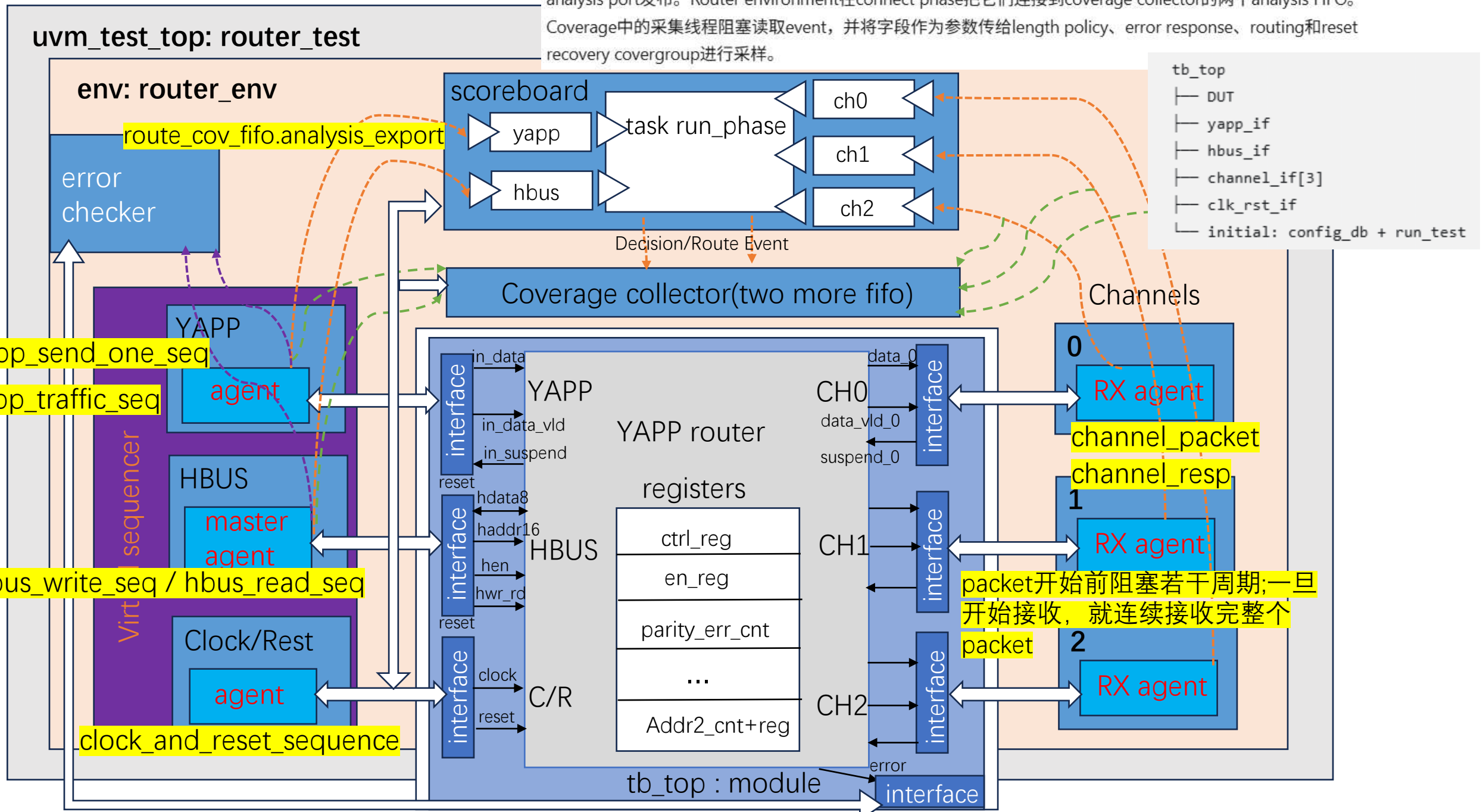
HTRANS

值	名称	含义
00	IDLE	没有传输
01	BUSY	突发传输中暂时停一下，不传有效数据
10	NONSEQ	一次单独传输，或突发的第一拍
11	SEQ	突发的后续拍

流水示例（无等待）	地址阶段	数据阶段
第1周期	A 的地址及控制	—
第2周期	B 的地址及控制	A 的数据
第3周期	无新传输	B 的数据

面试考点	速记
核心特点	地址、数据分阶段；**A的数据与B的地址可同时传**
接收有效地址	上升沿满足 **HSEL=1、HTRANS[1]=1、HREADY=1**
插入等待	**HREADY=0** 延长当前数据阶段，同时阻塞后续地址阶段
与 AXI 握手区别	无 VALID/READY 配对；**HTRANS 表示传输，HREADY 控制推进**
AHB-Lite	单 Master，省去多 Master 仲裁

Scoreboard在完成配置与输入包关联后生成decision event，在完成输入输出匹配后生成route event，通过两个analysis port发布。Router environment在connect phase把它们连接到coverage collector的两个analysis FIFO。Coverage中的采集线程阻塞读取event，并将字段作为参数传给length policy、error response、routing和reset recovery covergroup进行采样。




```
tb_top
├── DUT
├── yapp_if
├── hbus_if
├── channel_if[3]
├── clk_rst_if
└── initial: config_db + run_test
```

项目	内容
DUT功能	单输入、三输出 Packet Router，带 HBUS 寄存器接口
输入协议	Header + Payload + Parity
路由规则	addr 0/1/2 → Channel 0/1/2
主要功能	长度控制的路由、错误处理、计数器
验证方法	Directed + Boundary + Constrained Random + Code Coverage + Functional Coverage
环境结构	YAPP/HBUS/Channel/Clock-Reset VIP + Virtual Sequence + Scoreboard + Coverage + Error Checker

这是一个单输入、三输出的可配置Packet Router UVM验证项目。输入Packet由Header、变长Payload和Parity组成，Header包含6 bit Length和2 bit Destination；HBUS负责配置最大包长、Router Enable、Counter和Interrupt。

环境包含YAPP输入Agent、HBUS Master Agent、三个Reactive Channel Agent和Clock/Reset Agent，通过Virtual Sequence协调配置、发包、输出响应、Backpressure和运行中Reset。

项目的重点是5路Analysis FIFO的配置感知型Scoreboard：它维护 `max_packet_size` 和 `router_en` 的Shadow State，根据输入Packet预测Forward/Drop及目标Channel，再比较Header、Payload和Parity。Coverage同时采集Monitor原始事务和Scoreboard关联后的Decision/Route Event。

```
YAPP packet_delay
        ↓
发送Header/Payload/Parity
        ↓
Packet进入DUT目标FIFO
        ↓
DUT输出data_vld
        ↓
Channel生成resp_delay
        ↓
Driver保持suspend=1
        ↓
Monitor统计实际stall_cycles
```

第3页：Verification Plan

Feature	Stimulus	Check	Coverage
基本路由	向addr 0/1/2发包	对应Channel输出，内容一致	`route × length`
长度边界	max=N，发送N−1/N/N+1	N−1、N转发；N+1丢弃	`max class × relation × action`
非法地址	addr=3	不应输出，错误计数/状态更新	`drop reason × action`
Bad parity	合法包注入错误parity	包继续转发，error/计数器更新	`parity × action`
HBUS寄存器	默认值、写入、读回	readback一致	`operation × register`
Backpressure	不同Channel插入stall	无丢包、无乱序、输入可被反压	`length × stall` 、 `channel × stall`
Reset recovery	idle/in-packet时reset	旧包作废，新包正常路由	`reset context × recovery result`
Random regression	随机地址、长度、间隔、压力	Scoreboard自动检查	根据coverage hole补定向测试

面试问题

Channel VIP为什么reactive?

观察data_vld，生成suspend响应，模拟接收端背压

Monitor怎么知道stall?

`data_vld && suspend` 期间逐拍计数

Reset怎么处理旧包?

Monitor丢partial transaction；Scoreboard reset epoch

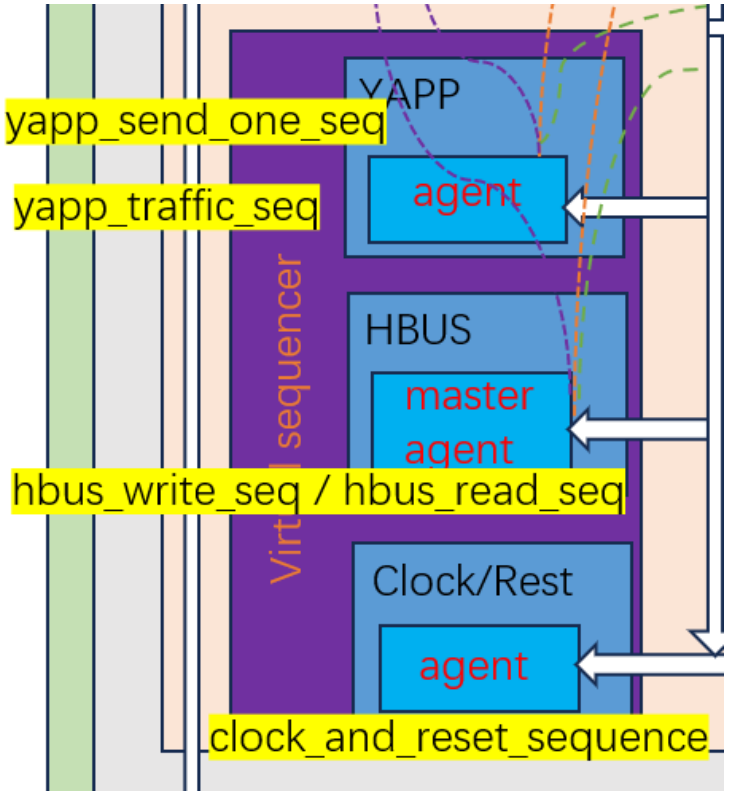
Error checker为什么独立?

Packet数据通路与sideband error职责分离

第4页：Stimulus组织

层级	负责什么	典型参数
Transaction	描述一个协议事务，默认约束合法	addr、length、payload、parity
UVC Sequence	产生基础协议流量或响应	packet count、gap、stall
Project Sequence	组合成Router专用流量	地址pattern、边界长度、错误类型
Virtual Sequence	协调多个VIP的时间关系	max size、target channel、pressure、reset时机
Test	选择VPlan场景并设置高层参数	使用哪个vseq、场景配置、checker开关

```
Test
  ↓ 选择场景和参数
Virtual Sequence
  ├─ YAPP traffic
  ├─ HBUS config/readback
  ├─ Channel response/backpressure
  └─ Clock/Reset
```



典型例子：长度边界测试

```
Test:
  max_size=20, target_channel=1

Virtual Sequence:
  1. HBUS写max_size=20
  2. 启动Channel 1 response
  3. 启动YAPP boundary sequence

Project Sequence:
  发送length=19、20、21

Scoreboard:
  19 → forward
  20 → forward
  21 → drop
```

随机测试

```
Virtual Sequence随机：
max_size、target_channel、pressure mode

Traffic Sequence随机：
packet length、gap、payload、parity概率

约束保证：
合法组合 + 受控错误注入 + 可复现seed
```

第一层：Raw Protocol Coverage 直接来自Monitor，证明接口上出现了什么：

```
YAPP Monitor    → addr、length、parity、payload pattern
HBUS Monitor    → read/write、register、readback data
Channel Monitor → output channel、length、实际stall cycles
```

第二层：Feature Coverage 来自Scoreboard关联后的语义事件，证明完整功能场景发生：

VPlan Feature	如何证明场景发生	对应Coverage
基本路由	addr0/1/2真实输入，对应Channel正确输出，覆盖不同包长	`packet_protocol_cg` + `routing_feature_cg`
长度策略	保存max-size快照，将Packet分类为below/equal/above，并确认forward/drop	`length_policy_cg`
非法地址/超长	记录drop reason，并确认action确实为drop	`error_response_cg`
Bad Parity	记录Bad Parity输入且Packet仍forward；Error由独立Checker检查	`packet_protocol_cg` + `error_response_cg`
HBUS寄存器	不同寄存器发生read/write，配置值被读回	`register_access_cg`
Backpressure	每个Channel实际观察到不同stall强度，并覆盖不同包长	`flow_control_cg`
Reset Recovery	Reset发生在idle/in-packet，Reset后第一个Packet正确路由	`reset_recovery_cg`
Random Regression	不是独立Coverage Feature，用于补充上述Coverage Hole	使用全部Covergroup

接口Cross

```
纯文本

HBUS Monitor更新配置镜像
        +
YAPP Monitor得到当前packet
        ↓
Scoreboard保存配置快照并计算length relation
        ↓
Decision Event
        ↓
同一个covergroup中sample并cross
```

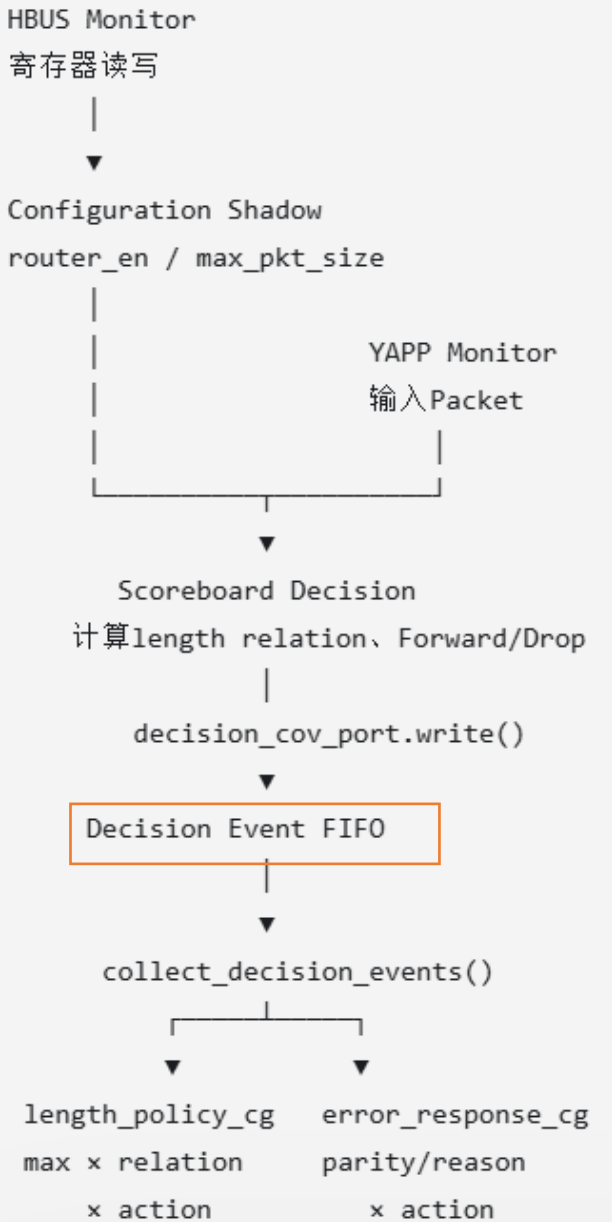
然后生成一个Decision Event：

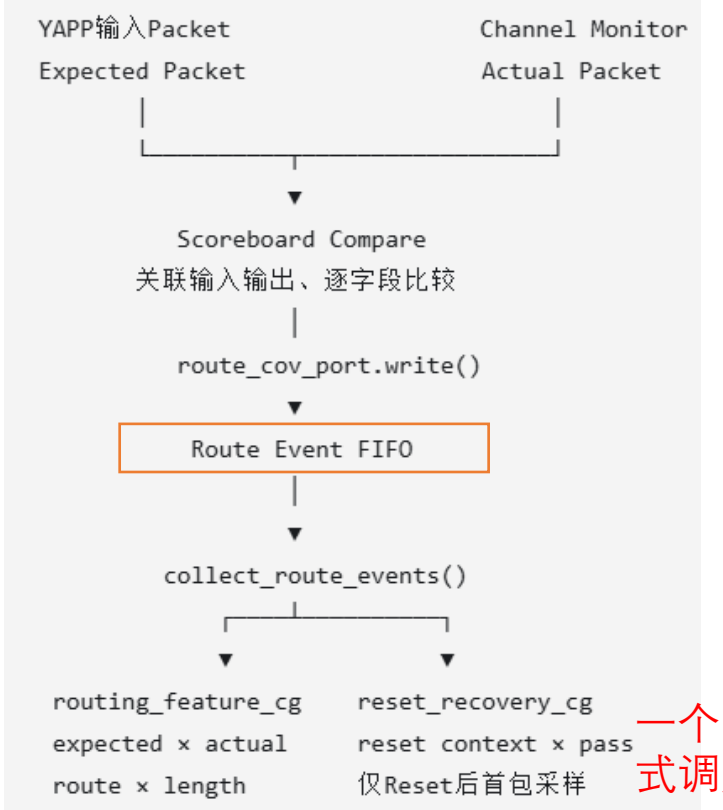
```systemverilog
enable_at_accept
max_size_at_accept
length_relation
expected_action
```

Coverage Collector收到后调用：

```systemverilog
length_policy_cg.sample(
    enable_at_accept,
    max_size_at_accept,
    length_relation,
    expected_action
);
```

Feature Coverage Event Flow　Scoreboard关联配置、输入与输出，生成语义Event并驱动Covergroup采样





Decision Event

```
Input
addr, length, parity

Configuration Snapshot
router_en, max_pkt_size

Prediction
length_relation
expected_action
drop_reason
```

Route Event

```
Request
request_addr, packet_length

Response
expected_channel
actual_channel
compare_pass
```

一个Event可以服务多个Covergroup，但每次采样都由Collector显式调用，不是Event到达后自动采样。

Event产生	Event传输	Coverage采样
Scoreboard完成关联后创建Event	Analysis Port发送，Analysis FIFO缓存	Collector收到Event后显式调用 `sample()`
Decision发生在Forward/Drop预测之后	`write()` 与采样线程解耦	一条Event可以采样多个Covergroup
Route发生在输入输出匹配之后	`get()` 等待下一条Event	不相关场景通过条件控制，不盲目采样

Event不会自动触发全部Covergroup。Coverage Collector取得Event后，选择相关字段并显式调用对应的 `sample()`。

第5页：Scoreboard与Reset

Scoreboard输入

```
纯文本

YAPP FIFO        : 输入expected packet
HBUS FIFO        : 更新配置镜像
Channel 0/1/2 FIFO: 实际输出packet
```

```
取得YAPP输入包
→ 保存当前reset_epoch和配置快照
→ 判断forward/drop
→ 发布Decision Coverage Event
→ 合法包：等待对应Channel输出或Reset
→ 比较addr/length/payload/parity
→ 发布Route Coverage Event
```

```
HBUS写0x1000 → max_pktsize_reg
HBUS写0x1001 → router_enable_reg
```

对每个输入包保存当时的配置，避免后续HBUS写操作改变这个包的判断含义。

Reset epoch

```
Reset前：packet_epoch = 2
Reset发生：reset_epoch = 3

packet_epoch != reset_epoch
→ 旧expected packet失效
→ 终止Channel FIFO等待
→ 不与Reset后的新包比较
```

第7页：RTL必记提示

DUT结构

```
纯文本

host_ctl   : HBUS寄存器、计数器、中断
port_fsm   : 解析输入、路由、错误判断
3×FIFO     : 三个Channel输出缓存
```

FSM

State	行为
HEADER_WAIT	等待header，检查地址与长度，锁存目的Channel
DATA_LOAD	写入header/payload/parity，处理FIFO full和输入暂停
DUMP_PKT	非法地址或超长时，消费剩余输入但不写Channel FIFO

背压传播

```
纯文本

Channel suspend
→ FIFO停止读取
→ FIFO full
→ hold
→ in_suspend
→ YAPP Driver暂停发送
```

TestBeach_bug

“Payload传输中插入Reset” “包中Reset”

Reset Recovery测试中，我在Packet传输期间插入Reset。原YAPP Monitor不感知Reset，可能将Reset前后的数据拼成一个Phantom Transaction，导致恢复包误匹配。修复时让Packet采集与Reset并行检测，Reset先发生就丢弃Partial Packet；同时Scoreboard在Reset时清空FIFO并恢复配置镜像，保证Reset后的Packet重新开始端到端检查。

```
Monitor保护：防止半包进入Scoreboard
Scoreboard保护：清除已经进入匹配流程的旧包
```

我还验证了“输入包已经完整接收，但输出因Backpressure尚未完成时插入Reset”的场景。原Scoreboard已经把Reset前Expected Packet取到局部变量，并阻塞等待Channel输出，单纯flush Analysis FIFO无法清除它，导致Reset后的Recovery Packet与旧包误匹配。最终我引入 `reset_epoch` ，并行等待Channel输出和Epoch变化；Reset先发生就终止当前匹配、清空FIFO并恢复配置镜像。

```
完整输入 + 输出背压 + 中途Reset
→ Expected已进入Scoreboard局部变量
→ flush FIFO无效
→ 新包与旧Expected误匹配
→ reset_epoch终止旧匹配
```

这和前一个Bug正好形成两层保护：

Reset发生阶段	环境问题	修复
输入Packet尚未采集完成	Monitor可能产生Phantom Transaction	Reset时丢弃Partial Packet
输入完成、输出尚未完成	Scoreboard局部Expected跨Reset残留	`reset_epoch` 终止旧匹配

Dut_Bug

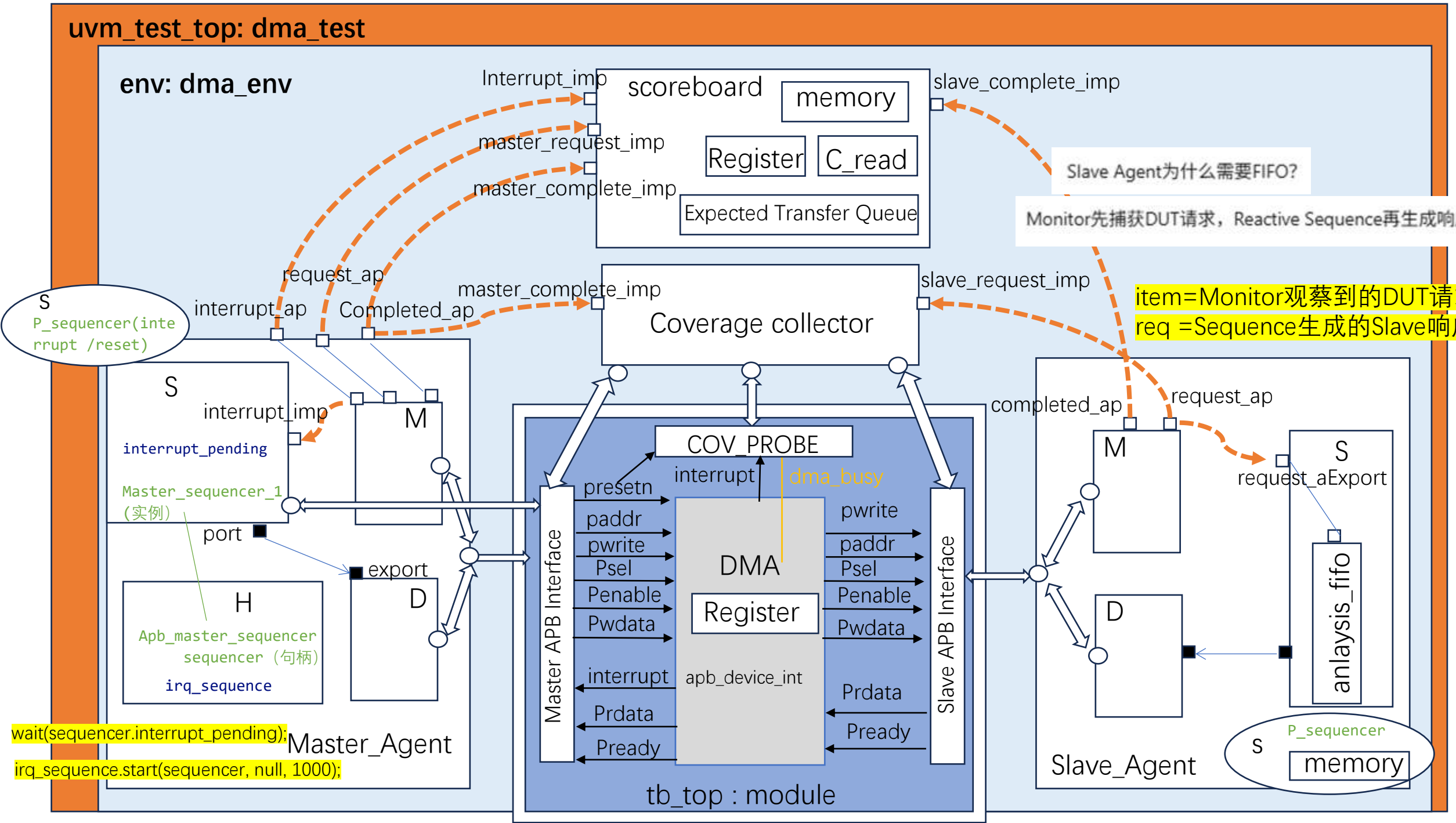
`router_en` RTL Bug | 面试小炒版

> 我通过HBUS将 `router_en` 配置为0，再向三个合法地址发送Packet。按寄存器定义，Router关闭后所有Packet都应该被丢弃，但仿真中三个Channel仍然收到了完整Packet，Scoreboard报出Unexpected Transaction。检查RTL后发现， `router_en` 判断被错误地嵌套在Length Error分支中，导致合法Packet绕过Enable检查并继续转发。最终将其定位为RTL与寄存器定义不一致的问题，并作为Known-fail隔离。

如果面试官问“为什么不是环境问题？”

> HBUS Monitor确认 `router_en=0` 已经成功写入，YAPP Monitor确认Packet正常进入DUT，三个Channel Monitor又确实观察到了完整输出，因此问题位于DUT的路由决策逻辑。

```
HBUS关闭Router
→ 三个合法地址发包
→ 预期全部Drop
→ 实际仍正常路由
→ Scoreboard报Unexpected Transaction
→ router_en误嵌套在Length Error分支
→ 合法包绕过检查
→ RTL/Specification Mismatch
```


uvm_test_top: dma_test
env: dma_env
Interrupt_imp
scoreboard
memory
slave_complete_imp
master_request_imp
Register
C_read
master_complete_imp
Expected Transfer Queue
Slave Agent为什么需要FIFO?
Monitor先捕获DUT请求，Reactive Sequence再生成响应
request_ap
master_complete_imp
slave_request_imp
item=Monitor观蔡到的DUT请求
req =Sequence生成的Slave响应
S
P_sequencer(inte
rrupt /reset)
interrupt_ap
Completed_ap
Coverage collector
S
interrupt_imp
M
interrupt_pending
Master_sequencer_1
(实例)
port
export
H
D
Apb_master_sequencer
sequencer（句柄）
irq_sequence
COV_PROBE
interrupt
dma_busy
presetn
paddr
pwrite
Psel
Penable
Pwdata
interrupt
Prdata
Pready
DMA
Register
apb_device_int
Master APB Interface
Slave APB Interface
pwrite
paddr
Psel
Penable
Pwdata
Prdata
Pready
tb_top : module
completed_ap
request_ap
M
S
request_aExport
D
anlaysis_fifo
wait(sequencer.interrupt_pending);
irq_sequence.start(sequencer, null, 1000);
Master_Agent
Slave_Agent
P_sequencer
s
memory

项目	关键词
DUT功能	单通道DMA；Direct、INIT、COPY；Interrupt/W1C
上游接口	APB Slave：CPU配置SRC、DST、LEN及命令
下游接口	APB Master：DMA主动访问统一Memory空间
数据搬运	固定32 bit；LEN=N → N次Read + N次Write
验证架构	Master Agent + Reactive Slave Agent + Scoreboard + Coverage
重点场景	长度边界、Overlap、非法操作、Wait-state、Busy
验证方法	Directed + Random + Functional Coverage +Code Coverage

验证环境数据流

```
Master Sequence → Sequencer → Driver → 上游APB → DUT
                                                  ↓
DUT下游请求 → Slave Monitor → Analysis FIFO
                                  ↓
                        Reactive Slave Sequence
                                  ↓
                  Slave Driver → PREADY / PRDATA
```

这个项目验证的是一个单通道DMA控制器。DUT上游是APB Slave接口，由CPU侧配置SRC、DST、LEN并下发INIT或COPY命令；下游是APB Master接口，由DMA主动访问统一地址空间中的Memory。数据粒度固定为32 bit，COPY长度为N时，会产生N次Source Read和N次Destination Write。

验证环境采用UVM架构：Master Agent模拟CPU配置和Direct访问，Reactive Slave Agent模拟下游Memory并插入PREADY Wait-state；上下游Monitor把事务发送给Scoreboard和Coverage Collector。Scoreboard维护寄存器Shadow Model、Reference Memory及Expected Transaction Queue，对地址、方向、数据、传输顺序和Interrupt W1C行为进行端到端检查。

`p_sequencer` 主要给业务Sequence提供类型化Sequencer句柄。Monitor检测IRQ后把pending、level和service count保存在Sequencer中，普通Sequence读取这些状态，等待中断处理完成后继续发送事务。Sequencer里保存的 `APB_VMINTF` 则主要服务于Reset测试，Sequence通过它观察时钟和PSEL/PENABLE状态，把presetn精确插入Idle、Setup或Access阶段。它不用于驱动IRQ，因为IRQ是DUT输出。

第5页：核心验证计划

功能	如何测试	Scoreboard如何检查
Reset/Register	Reset后读写SRC、DST、LEN	检查默认值、写入和读回
Direct	Direct写后读，多种数据和地址	检查下游地址映射、方向、数据和上游读回
INIT	配置SRC/LEN，写INIT值	生成 `Write SRC+i` 队列，检查数据和数量
COPY	配置SRC/DST/LEN并预填Source	生成Read/Write交替队列，检查COPY数据
Interrupt	制造Done、Overlap、非法LEN和非法操作	检查INT读回、IRQ和W1C
Wait-state	延迟上下游PREADY	Scoreboard检查不丢不重，SVA检查信号稳定
Busy/Random	DMA期间CPU访问及随机组合	检查DMA队列和数据没有被破坏

Wait-state测试同时覆盖上下游：上游等待由DUT随机PREADY延迟产生，下游等待由Reactive Slave Sequence控制；Coverage统计等待周期，SVA检查上游信号稳定，Scoreboard检查等待后没有事务丢失、重复或数据错误。

Busy测试在启动DMA INIT后不等待完成，立即发起Direct、寄存器及非法CPU访问；通过内部busy probe确认请求确实落在DMA运行期间，再由Scoreboard检查原有DMA下游队列、地址和数据没有被破坏。

Random Test在合法安全边界内随机组合Direct、INIT、COPY、非法长度及Reserved访问，默认每个Seed执行40次；多Seed回归扩大状态组合，由Scoreboard持续检查DMA队列、Reference Memory和Interrupt状态，并支持通过Seed复现失败。

第4页：Stimulus组织

层级	负责什么	DMA项目中的实现/参数
Transaction	描述单笔APB事务	`addr、write、data、addr_delay、ready_delay`
Base Sequence	封装基础协议操作	`cpu_read/write、run_init、run_copy、reset注入`
Reactive Slave Sequence	根据DUT请求产生响应	Memory读写、`PRDATA`、`PREADY delay`
Project Sequence	组合DMA功能场景	Direct、INIT、COPY、Interrupt、Busy、Reset、Random
协同机制	协调Master、Slave和IRQ	Slave FIFO、`p_sequencer`状态、Interrupt Handler
Test	选择Sequence并配置场景	设置Master/Slave default sequence、Seed、Random次数

本项目未使用Virtual Sequence：Master负责CPU访问，Slave通过FIFO被动响应DUT请求，IRQ由独立Handler在Master Sequencer上启动高优先级Sequence。

DMA COPY地址关系测试

```
Test：
dma_apb_copy_test

Master Sequence：
dma_apb_copy_sequence

Slave Sequence：
apb_slave_basic_sequence
```

Stimulus流程

1. Direct Write预填Source Memory
2. 配置DMA_SRC、DMA_DST、DMA_LEN
3. 写DMA_COPY启动搬运
4. Slave Agent响应Source Read和Destination Write
5. 等待Done Interrupt及W1C清除

```
LEN：1、4、16、33
地址关系：
SRC = DST
前向/后向Overlap
前向/后向Adjacent
Disjoint
```

检查内容：

- Read/Write必须交替；
- Source/Destination地址递增正确；
- COPY写数据等于前一笔Read返回值；
- 下游传输数量等于`2 × LEN`；
- Reference Memory最终状态正确；
- Overlap和Done Interrupt符合预期。

第4页：Scoreboard重点

Port	作用
Master request_ap	Setup阶段；Direct提前建期望、预测Invalid-op和Overlap
Master completed_ap	寄存器读写、INIT/COPY命令、INT读回和W1C
Master interrupt_ap	IRQ边沿通知Sequencer、Handler及Scoreboard
Slave request_ap	下游Setup请求送Reactive Sequence和Coverage
Slave completed_ap	Scoreboard检查实际下游地址、方向、数据和顺序

Scoreboard有四个输入。Master Request在上游Setup阶段到达，用于提前建立Direct下游预期，以及预测非法操作和COPY重叠；Master Complete用于更新SRC、DST、LEN寄存器模型，检查寄存器和Direct读回，并在INIT/COPY命令完成后生成预期下游队列；Slave Complete负责将DUT实际完成的下游事务与队列逐笔比较；Interrupt接口记录IRQ引脚状态。

核心检查链

```
Master Request/Complete
→ 生成Expected Slave Queue
→ Slave Complete提供Actual
→ 比较Direction、Address、Data、Order
```

内部模型

```
Register Model: SRC、DST、LEN
Reference Memory: 预测Memory内容
Expected Transfer Queue: 预期下游事务
COPY Read Data Queue: 保存COPY读回数据
Expected Interrupt: 预测4位中断状态
```

一、Coverage整体结构

覆盖层次	Covergroup	主要覆盖内容	对应验证计划
输入场景	`cg_master_access`	地址区域、读写方向、LEN边界、数据类型	Register、Direct、非法访问
输入场景	`cg_dma_command`	INIT/COPY、LEN分类、SRC/DST关系、配置是否完整	INIT、COPY、Overlap
中断行为	`cg_interrupt`	Done、Overlap、Invalid Length、Invalid Op、组合状态、W1C	Interrupt、异常场景
输出行为	`cg_slave_request`	DUT发起的下游读写、访问Source/Destination、写数据类型	Direct、INIT、COPY
输出行为	`cg_slave_completion`	实际完成的下游读写、地址目标和数据类型	Direct、INIT、COPY
时序状态	`cg_apb_wait`	上下游、读写方向、等待周期	Wait-state
时序状态	`cg_reset`	Idle、Setup、Access阶段Reset	Reset
内部状态	`cg_probe_state`	IRQ与DMA Busy的高低及跳变	Interrupt、Busy
组合压力	`cg_busy_attempt`	DMA Busy时CPU访问的[ ]读写方向	Busy/Random

建议把Coverage重新理解成三层：

Master事务告诉我“配置了什么、下了什么命令”；
Slave事务告诉我“DMA实际发出了什么访问”；
Virtual Interface告诉我“这个访问经历了什么时序”；
COV_PROBE告诉我“DUT当时处于什么内部状态”。

4. 关键Cross Coverage

```
Command × LEN
COPY LEN × SRC/DST Relation
APB Side × Read/Write × Wait
Busy × Address Region × Read/Write
```

只交叉存在功能关联的维度，避免Coverage空间无意义膨胀。

步骤	关键词提示 Tb_Bug
测试场景	Direct Write；下游PREADY快，上游PREADY慢
异常现象	slave_complete先到，但expected_slave_q为空
UVM报错	Unexpected downstream transfer
波形定位	对齐Master Setup、Master Completion、Slave Completion
根本原因	Scoreboard等上游完成后才生成期望，预测时机过晚
问题分类	Testbench建模问题，不是RTL数据错误
解决方法	Master Setup提前建期望；Slave Completion匹配；Master Completion检查最终读回
验证结果	不再依赖上下游完成顺序，Wait-state场景能够正确匹配

Case 1 | Direct访问出现Unexpected Transfer

Direct访问的难点不是数据比较，而是上下游存在独立Wait-state，Scoreboard必须在Setup阶段提前建立因果关系。

步骤	关键词提示 Dut_Bug
测试场景	写入合法LEN后，读取DMA_LEN
异常现象	LEN读回正确，但IRQ意外拉高
Scoreboard报错	DMA_INT expected=0，actual=0x4
波形定位	PWRITE=0、PWDATA=0、LEN Read完成时中断置位
根本原因	RTL使用Read事务中无意义的PWDATA判断LEN是否合法
缺失条件	非法长度判断没有限定PWRITE
修复建议	仅在有效写握手完成时判断LEN：PSEL && PENABLE && PREADY && PWRITE
当前规避	LEN Read暂时将无意义的PWDATA驱动为合法非零值

LEN Read误触发Invalid-Length中断

DUT功能 地址空间

PADDR[31:28]	功能
0x0	Direct Access
0x1	SRC
0x2	DST
0x3	LEN
0x4	INIT
0x8	COPY
0xF	INT
其他	Invalid Operation

三条功能路径

合法LEN: 1~16
IRQ输出: |dma_int
清除方式: W1C

Direct: 上游访问直接转发到下游

INIT: 写SRC+i = 固定INIT数据

COPY: Read SRC+i, 再Write DST+i

COPY为什么两阶段？
APB一次只能完成一笔单方向事务，必须先读再写。

中断状态

Bit	含义
0	DMA Done
1	COPY Overlap
2	Invalid Length
3	Invalid Operation

第7页：仿真与回归

工具和文件

名称	作用
VCS / `simv`	编译和运行仿真
Exit Code	仿真进程是否正常结束
VDB	Coverage数据库
URG	合并VDB并生成Coverage报告
FSDB	Verdi波形文件
VPD	DVE波形文件

回归流程

VCS一次编译全部Test
运行定向Test
Random Test运行多个Seed
check_run.sh检查结果
URG合并VDB
生成HTML Coverage报告

Pass条件

Simulator exit code = 0
UVM_ERROR = 0
UVM_FATAL = 0
无Assertion Failure
Scoreboard队列和中断状态全部清空

引入RAL后可以规范描述：

- `DMA_SRC`：28位R/W；
- `DMA_DST`：28位R/W；
- `DMA_LEN`：5位R/W；
- `DMA_INT`：W1C；
- Reset值和字段访问权限；
- Frontdoor寄存器Sequence；
- Mirror和Readback检查。

典型结构：

RAL Sequence
→ Register Model
→ APB Adapter
→ Master Sequencer/Driver
→ DUT

Master Monitor
→ RAL Predictor
→ 更新Mirror

Shell脚本更适合环境配置、流程执行和回归控制；Makefile更适合定义编译、运行、覆盖率等目标并管理依赖。二者并非必须同时使用，小项目用Shell即可，较完整的验证环境通常以Makefile作为统一入口，再调用Shell或Python脚本完成具体任务。